Tutorial #3 - Reversi

Introduction

Welcome to the third of our AppGameKit Studio tutorials, aimed at those of you who are just starting to explore the world of tier 1 (BASIC) coding. As before, we'll be looking at another different game genre, learning about various common coding techniques that are often used in game making, and bringing them all together to create a simple template of a game – in this case the game *Reversi*.

This tutorial will expand on what we previously learnt from our Pong and Tanks games, all techniques can be used in your future projects. Once again, you'll be guided through each of the steps clearly, and be encouraged to change various values and lines of code so that you can learn first-hand how simple changes can affect the code.

The entry level for these tutorials expects some initial understanding of the students in the use of AppGameKit Studio. For more beginner level tutorials they can read the AppGameKit Studio Online Guides and watch the YouTube Videos.

Students are also encouraged to join the Official AppGameKit Studio forums. There's a newbie section where new users will be helped by many experienced and friendly developers who use AppGameKit Studio all the time.

Objectives

These tutorials aim to teach coding and game development using a combination of examples and action-based teaching. You will learn from studying the code snippets, reading and understanding the comments and gain an additional understanding of each section through a number of simple exercises aimed at reinforcing each lesson and will help build your confidence by demonstrating the ease with which the code can be altered.

This third tutorial aims to introduce the concepts of basic coding and is designed to lay the groundwork of future developments. During this tutorial we will:

- Learn more about variables and learn how to use them effectively
- Learn a basic search technique
- Introduce different ways to manipulate sprites
- Learn how to translate screen coordinates into usable information
- Gain an understanding of a form of basic AI by creating a effective opponent

At any point, examples of almost all coding commands can be accessed from the IDE (integrated development environment) simply by either clicking directly to the left of the command and pressing F1 on the keyboard. You're encouraged to learn this method from the beginning as it will prove to be an invaluable reference guide and is a resource often used even by coders who are very familiar with AppGameKit Studio.

You can also access a full command reference from the IDE. By clicking Help on the top tool bar, you can gain access to a number of reference sources, including video tutorials, a list of useful keyboard shortcuts as well as the command reference.

Each section of code is available as a separate project which can be simply loaded into the IDE, but it it recommended that you learn to type the code into the IDE, as this will help to develop a familiarity with AppGameKit Studio overall and will promote confidence in using the functions of the IDE as well as reinforcing the learning process.

Now let's move on…

As with our previous tutorial for our Tanks game, we will be working with the full code and from the start. We'll then breakdown this code and discuss the various sections, routines and Functions in a clear way, whilst providing small EXERCISEs and challenges to help you gain a greater understanding and familiarity with AppGameKit Studio.

As before, we'll load our project from the tutorial folder, in this case, Reversi.

For those of you who don't know the game Reversi (Sometimes called Othello), it is a strategy board game for two players, played on an 8×8 board. There are sixty-four identical game pieces, which can either be Black or White. Players take turns placing pieces of their own colour on the board. During play, any pieces of the opponent's colour that are in a straight line, either horizontal, vertical or diagonal, and is between the piece just placed and another piece of the current player's colour are replaced with the current player's colour. You can capture pieces in multiple directions each move.

The object of the game is to have the majority of pieces changed to display your colour when the last playable empty square is filled.

Load up the project and compile and run it. You should be presented with a screen similar to this one below. By default, the human player plays the black team and goes first.

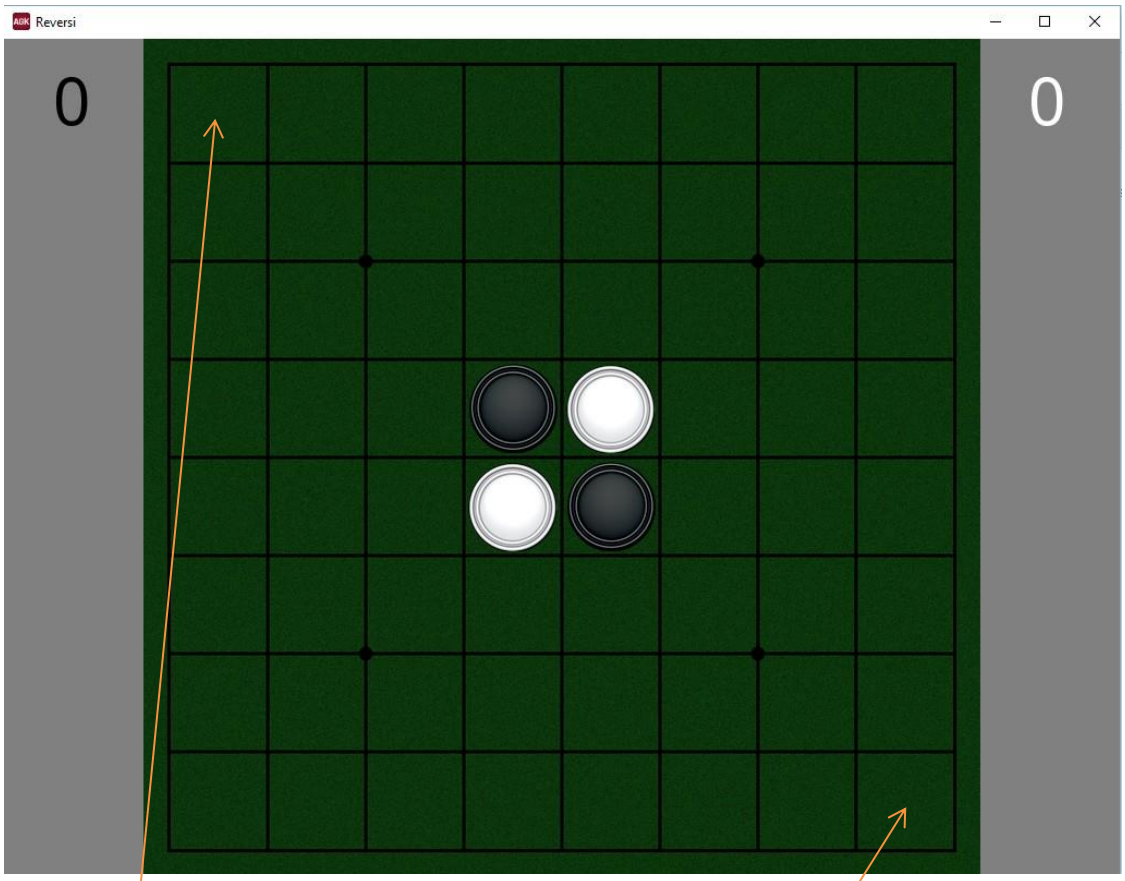


Fig 1 – Basic Board

As you can see, the board is divided up into an 8x8 grid, for the purposes of our game, you should consider the top left-hand square as position 1,1 and the bottom right hand square as position 8,8.

The first value is the across the board position, and the second value is the down position. If you move your mouse around, you'll see one of two things. Either, a black piece attached to your cursor, or a dark red piece.

Let's look at the dark red piece first. Move your mouse around until your game looks like this. You are trying to place a piece at position 4,3 (4 squares across and 3 squares down). You're seeing a dark red piece because this is an illegal move because placing a black piece here does not trap a white piece between two of your pieces already on the board.

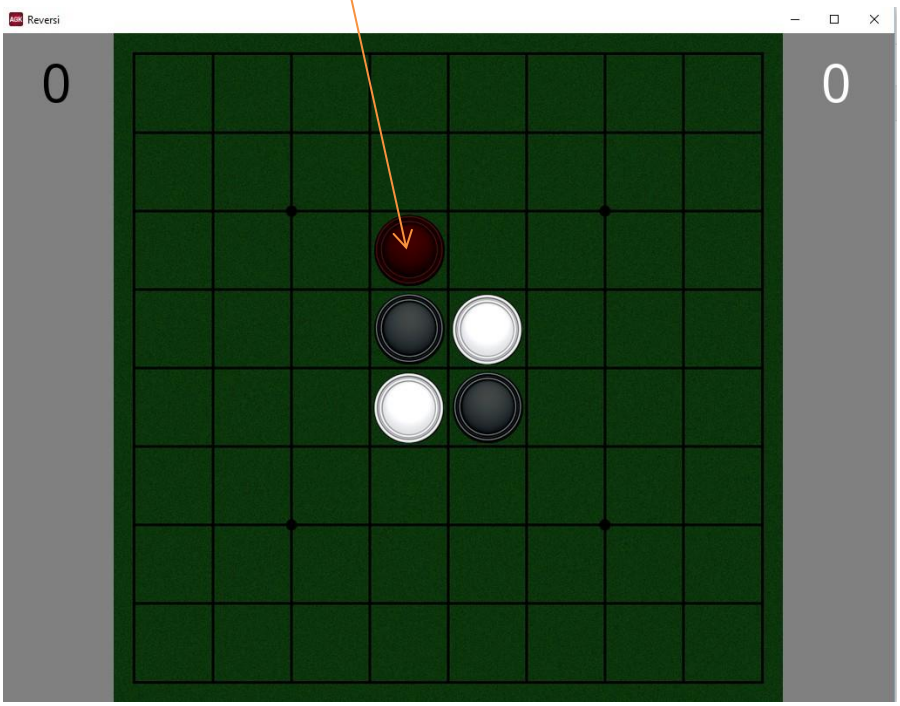


Fig 2 – An illegal move.

Now let's look at a legal move;

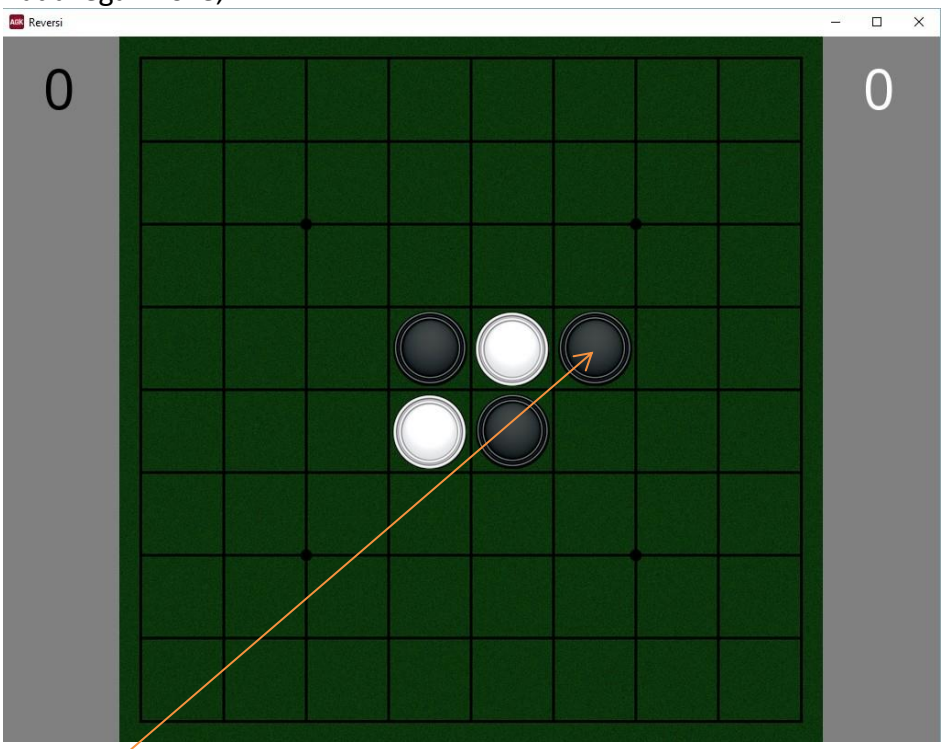


Fig 3 – A legal move.

Once again move your cursor around until your game looks like this. You are trying to place a piece at square 6,4. This would be a legal move, because you are trapping the white piece at 5,4 between your piece at 4,4 (already on the board) and the new piece placed at 6,4.

Let's left click and make our move at position 6,4. You will briefly see a change to the image below. (And we mean briefly as the computer will make it's move instantly, so fast in fact that when we study the code you'll see we've had to slow it down).

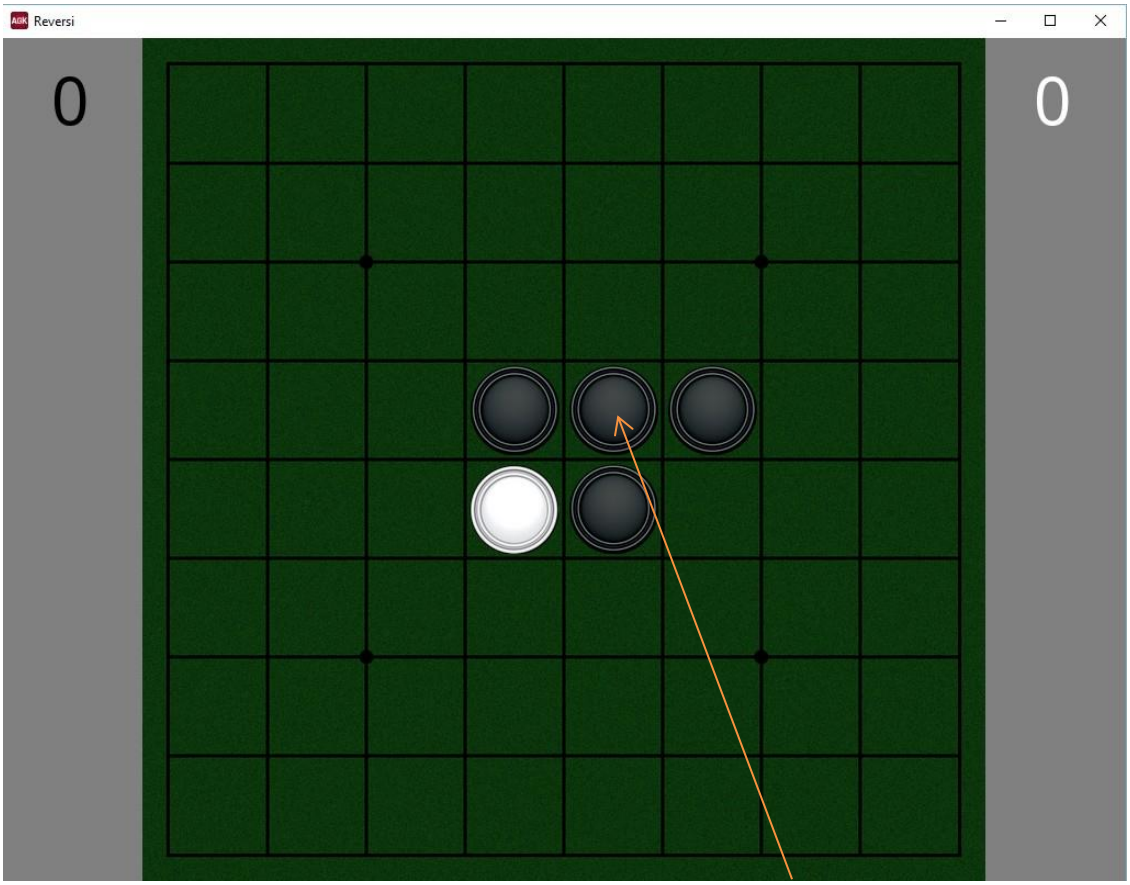


Fig 4 – Player has place a black piece at 6,5 and so 6,4 becomes black.

Hopefully you understand the rules, now let's look at how this all works.

First, we'll break the project down into what you'd expect to see if you had created the Reversi project from scratch.

```
// Project: Reversi
// Created: 2017-04-13

// Show all errors
SetErrorMode(2)

// Set window properties
SetWindowTitle( "Reversi" )
SetWindowSize( 1024, 768, 0 )
SetWindowAllowResize( 1 ) // Allow the user to resize the window

// set display properties
SetVirtualResolution( 1024, 768 ) // Doesn't have to match the window
SetOrientationAllowed( 1, 1, 1, 1 ) // Allow both portrait and landscape on mobile devices
SetSyncRate( 30, 0 ) // 30fps instead of 60 to save battery
SetScissor( 0,0,0,0 ) // Use the maximum available screen space, no black borders
UseNewDefaultFonts( 1 ) // Since version 2.0.22 we can use nicer default fonts
```

By now this should all be very familiar. If there's anything here you don't understand, go back and look at the earlier tutorials or use the help function to read more about these commands.

So, we've set up our project. Now let's look at the next bit of relevant code. This is another section

that sets up the basic game.

```
//We'll start off by changing the screen background colour from the default black
//This will make it easier to see some of our game details.
SetClearColor(128,128,128)
//Let's set up our sprites.
//As before, we'll set up a global variable to contain the values.
//We only need one for the board
Global playingboard=1
LoadImage(playingboard,"board.png")
CreateSprite(playingboard,playingboard)
//Position our sprite in the middle of the screen
SetSpritePosition(playingboard,128,0)
//Load black and white images
Global black=2
Global white=3
//Indicate whose turn it is and where they might place their piece
Global marker=2
//We also might want to make the game more or less challanging, so we'll add a
//Difficulty flag.
Global howdifficult=1
LoadImage(black,"black.png")
LoadImage(white,"white.png")
//We also need a sprite for the marker
//As Black goes first, we'll make it Black
CreateSprite(marker,black)
SetSpriteColorAlpha(marker,150)
```

Again, most of this should be pretty straight forward, but we do introduce two new commands which we've not covered before.

```
SetClearColor(128,128,128)
```

This will clear everything on screen and change the screen colour to a light grey. If you recall, colours are made up of three elements, a Red element, a Green element and a Blue element. This is often shortened to *RGB*. The values of each element runs from 0 (no amount of that colour) to 255 (the most of that colour that there can be), by mixing these colours together, we can create almost any shade or colour you can imagine.

EXAMPLE: If you wanted a pure green background you would use.

```
SetClearColor(0,255,0)
```

EXERCISE: *Change the screen background to pure Red and then to pure Blue. You'll need to run the project between each change. Once you've completed this EXERCISE feel free to experiment with other values.*

Let's take a look at the second new command.

```
SetSpriteColorAlpha(marker,150)
```

We've used something similar to this and the `SetClearColor` before, but first let's look at `SetSpriteColorAlpha(sprite,alpha).` In this case the sprite is called *marker*, and is the sprite we'll use to show where the player might place a piece. (Do you remember that this can change colour in our game?)

`SetSpriteColorAlpha` sets a sprites alpha value, which is how transparent a sprite is and similar to the colour commands has a range of 0 to 255. A value of 255 sets a sprite to fully opaque (not transparant), while a value of 0 would be completely transparent and would in effect be

invisible. You can compare both of these commands to this one;

```
SetSpriteColor(sprite,red,green,blue, alpha)
```

We've used this one before and it has a similar effect in that we can change not just the alpha of a sprite, (The fifth value), but also it's red, green or blue colour value (Just like the SetClearColor settings.).

Okay, we won't cover much more of the above code as you should have a decent understanding of it by now from our previous projects, but there is one important variable we've set up here, and that's *howdifficult*. You most likely won't be surprised to discover that changing this value changes the challenge the computer opponent will provide. This has 3 settings.

1=Provides a basic challange and won't use strategy.

2=Provides a challange and will attempt to take advantage of the edge squares

3=Plays a full game and will attempt to take advantage of all opportunities. This is a very challenging difficulty level.

More of that later when we look at how the computer thinks. Now let's take a look at setting up the board.

```
//Set up a new type for the board
type pieces
        //This is where we'll store the sprite position
        x as float
        y as float
        //Is the square in use?
        side as integer
        sprite as integer
        //We use this to decide when to change the colour of the piece
        flagged as integer
endtype

//We're using another muliple dimensional array here.
//This allows use to store information in a very similar way to the board layout
//Apart from using two coordinates to reference the array
//This is identical to the arrays we used in the first tutorial
dim board[8,8] as pieces

//We set up the pieces here
nextsprite=100
//Let's store the temporary screen coordinates here
tempx=156
tempy=28

//We'll use 2 For - Next loops to reference our array
for across=1 to 8
        for down=1 to 8

                board[across,down].sprite=nextsprite
                board[across,down].x=tempx
                board[across,down].y=tempy
                //For now we'll set all space on the board as empty
                board[across,down].side=0
                //The sprite needs an image, so we'll apply the black piece for now.
                CreateSprite(board[across,down].sprite,black)

        SetSpritePosition(board[across,down].sprite,board[across,down].x,board[across,down].y
)
        //We can hide a sprite by setting is to non visible. This allows us to show a blank
space
        // simply by hiding the sprite.
                SetSpriteVisible(board[across,down].sprite,0)
                //Move the variable with the sprite to be used value along 1
```

```
                inc nextsprite,1
                inc tempy,90
        next down

           tempy=28
           inc tempx,90
next across
```

Right then, let's look at this code, again you should find this familiar, so we'll recap what we've done, but in general we'd hope that you will see some similarities with how we designed our map in the Tanks game.

Let's compare: We scan across and down our grid using For – Next Loops.. We run down the grid first, and then across. We set every square to be empty (`board[across,down].side=0`), by default we're setting every sprite to be black so it has an image, but we'll change this in a bit. We set the sprite postion as normal, and so only pieces in use on our board can be seen, we'll make it invisible. (`SetSpriteVisible(board[across,down].sprite,0)`)

In our Tanks tutorial we calculated our screen position, for this one, we'll update the values *`tempx`* and *`tempy`*. Apart from that, we are using a very similar principal.

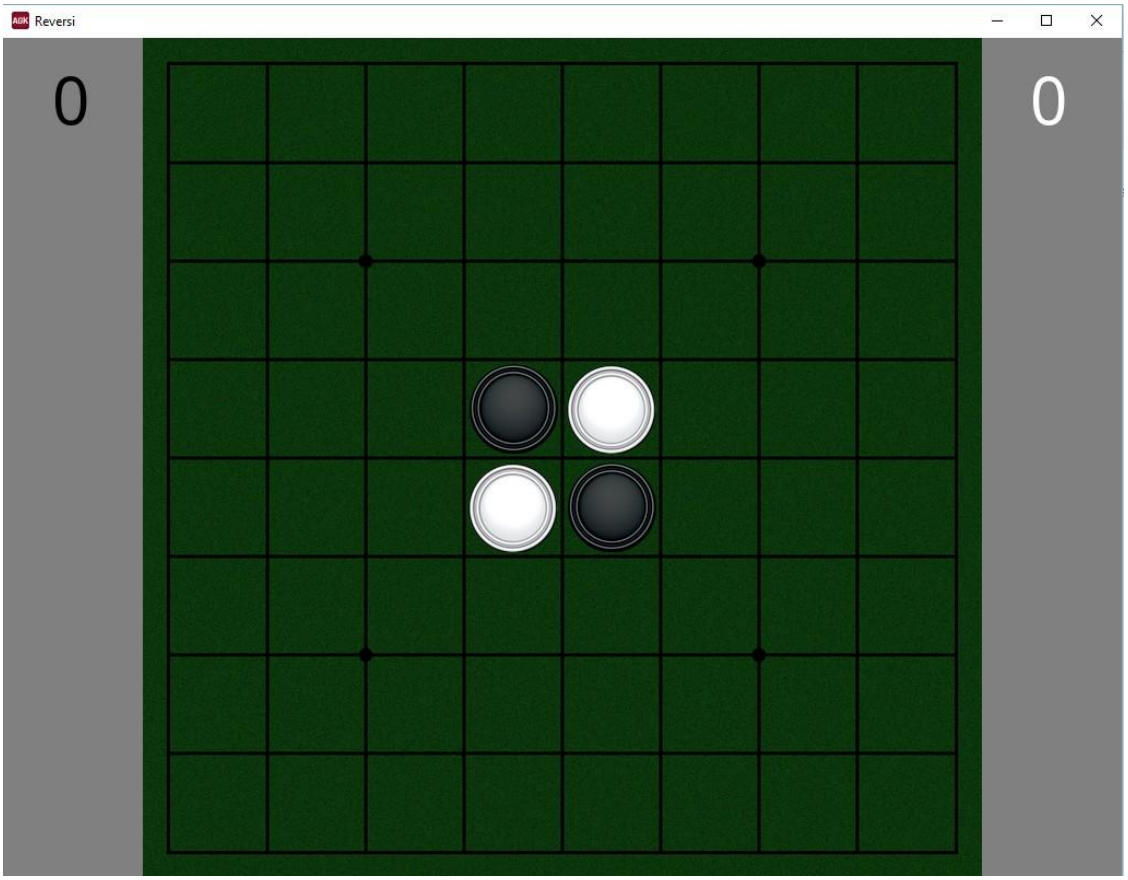
Now we'll take a look at our first Function. This clears the board and sets it up for a new game.

```
//As we will need to reset the game at some point, we might as well create
//a function to do it for us.
resetgame()

And the function is:

Function resetgame()

        for across=1 to 8
        for down=1 to 8
                board[across,down].side=0
                SetSpriteVisible(board[across,down].sprite,0)
        next down
        next across

        board[4,4].side=2
        board[5,4].side=3
        board[4,5].side=3
        board[5,5].side=2

EndFunction
```

This should make a lot of sense, as the code is more or less identical to the code we used to create a map in the Tanks game. We clear the board (the array) and change all the squares to zero (not held by anyone).

We then set the board squares to contain a piece, two as black using 2, and two more as white with the value 3. (We use 2 and 3 as they are the same numbers as the image numbers for black and white, and it will be easier later on to change the pieces when one side or the other wins them.).

Compare the array coordinates with this image, and you should see how we've set up the board. (count the squares across and down.).



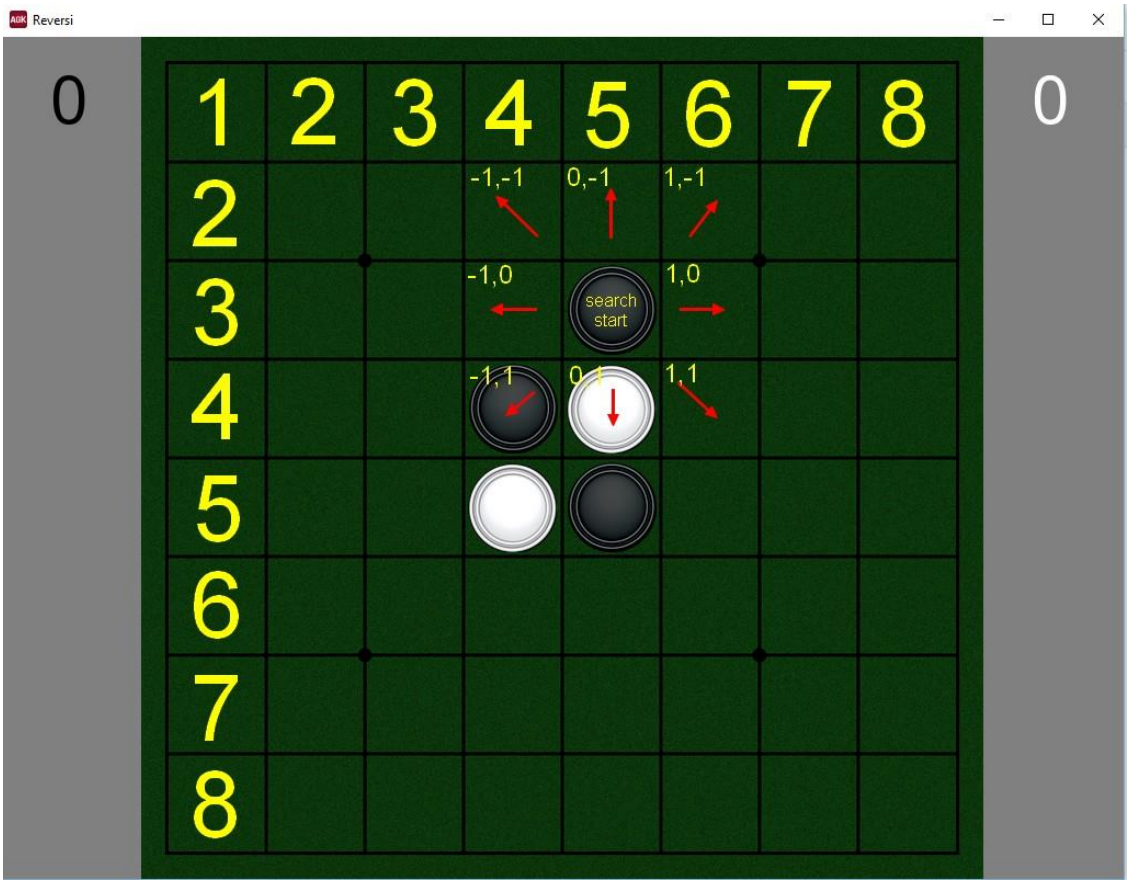
In a while we'll show you where the images change, but we need to do a little more set up first.

```
//Now let us set up a type for each player
//This one only needs a couple of different choices.

Type whoseplaying
       iscomputer as integer
       score as integer
EndType

//And create an array
//There are only 2 possible players
Dim player[2] as whoseplaying

//We decide who is being played by a human or a computer
player[1].iscomputer=0
player[2].iscomputer=1

//We'll need to set up someway to display the score, let's add some text objects.
//You should recognise these commands from previous tutorials
CreateText(1,"0")
SetTextSize(1,80)
SetTextPosition(1,45,15)
//The first player uses black pieces
//Let's change the colour of the text for this player to black
//So we know what's going on.
SetTextColor(1,0,0,0,255)
//Text 2 doesn't need a colour set as white is the default.
CreateText(2,"0")
SetTextSize(2,80)
SetTextPosition(2,940,15)

//We will need to store the position of the square under the cursor
//This is used when calculating if the player can go.
Global squareacross
Global squaredown
```

We've covered all of these commands before and we won't go into more detail now. Don't forget, if you're confused about anything, you can look at the previous tutorials or use the F1 help when your cursor is over a command you want more information about.

Let’s look at some A.I. To work out our moves we'll simply scan the grid from the point we want to place a piece. We need to do this in 8 directions.

Look at the image below, this demonstrates how the board is laid out and how our search direction will work.



Let's look at the grid above. We are placing our piece at 5,3. We will need to search in 8 directions as indicated by the arrows. We start all of our searches by looking up the grid. If we are starting at square 5,3 the square above that would be 5,2. Which is 5+0,3-1 or as you can see from the figures in each of the squares with red arrow, 0,-1. The similar thing is true if we wanted to search the square to the top-right of our starting square. Starting from 5,3, this would be 5+1,3-1 or simply 1,-1

We'll look at the search in more depth in a few moments, but for now, we'll set up a new array that will contain all the search information we need. Our search starts at the first element of our array and moves clockwise from element 1 through to element 8. Compare the values we assign below with the image above for a greater understanding.

```
//Now let's set up another Type
//This one will be used to supply coordinates to scan the board.
//We'll need to scan the board in 8 directions, horizontally, vertically and diagonally

type where
       across as integer
```

```
        down as integer
endtype

dim search[8] as where

//1 will be up
search[1].across=0
search[1].down=-1
//2 will be up and right
search[2].across=1
search[2].down=-1
//3 will be right
search[3].across=1
search[3].down=0
//4 will be down and to the right
search[4].across=1
search[4].down=1
//5 will be down
search[5].across=0
search[5].down=1
//6 will be down and left
search[6].across=-1
search[6].down=1
//7 will be left
search[7].across=-1
search[7].down=0
//8 will be up and left
search[8].across=-1
search[8].down=-1
```

Now let's look at our main game loop, again this should be familiar as it ties in with our previous tutorials rather nicely.

```
//Main game loop
Do


        setimages()

        for whoseturn=2 to 3

                SetSpriteImage(marker,whoseturn)

                if player[whoseturn-1].iscomputer=0
                        //Let's see if the player can actually move
                        //If not, it's game over
                        test=computerplayer(whoseturn,0)
                        do
                        hasmoved=humanplayer(whoseturn)
                        if hasmoved=1 or test=0
                                setimages()
                                counttiles()
                                 Sync()
                                Exit
                        Endif

                        Sync()

                         loop
                EndIf

                //If the player is the computer take it's turn
                if player[whoseturn-1].iscomputer=1
                        //Flag the computers turn as 'real'
                        //We will also return the pieces captured
                        //In case the computer could not go.
                        test=computerplayer(whoseturn,1)
                        setimages()
                        counttiles()
                        Sync()
                EndIf

                sleep(1000)

                //If test=0 (returned from our game loops)
                //No one can go, and the game is over.
                If test=0
```

```
                        If player[1].score>player[2].score
                        Print("Black has won!")
                        Endif

                        If player[1].score<player[2].score
                        Print("White has won!")
                        Endif

                        If player[1].score=player[2].score
                        Print("The game is a draw!")
                        Endif

                Endif

        Next whoseturn

Loop
```

Let's break this down. We start our `Do - Loop`. Then the first thing we do is call a function called `setimages()` - which looks like this.

```
Function setimages()

        for across=1 to 8
        for down=1 to 8
                If board[across,down].side<>0
                        SetSpriteImage(board[across,down].sprite,board[across,down].side)
                        SetSpriteVisible(board[across,down].sprite,1)
                Endif
        next down
        next across

EndFunction
```

Let's examine this. We scan our array, down and then across. If we find a square that has a piece in (I.E. The array reference does not contain a 0, and therefore must contain either 2 or 3 (2 for black, 3 for white), we'll set the sprite image to be the same as the value contained within board[across,down].side, because as we discussed earlier, we know that the image numbers match the value we've chosen to represent each players colours.

We also make this sprite visible so we can see it, and of course we'll use;

```
SetSpriteVisible(board[across,down].sprite,1)
```

We then finish our loops and return from the Function. By calling this after every turn, we ensure that our board is always up to date.

Right, let's see what's next. We run a new loop called *whoseturn* and we'll run this from 2 to 3, because if you've not guessed, that's the number that represents each player's pieces. You'll notice that we sometimes need to reference the array that contains the player information and this runs from 1-2, so we'll simply reduce the reference by -1.

E.g. `if player[whoseturn-1].iscomputer=0`

Because, *whoseturn -1* will either equal 1 or 2. We'll assume that this is the basic game, and that the human player is going first.

Right then, now we actually use the computer AI routine to do something rather clever, we can search all the squares as if we were the computer, but we have a special flag with defines if the search is a real computer move or a test and have the Function called computerplayer(?,?) return

the best possible move.

```
//Let's see if the player can actually move
//If not, it's game over
test=computerplayer(whoseturn,0)
```

So, that's whoseturn (which player), and 0 for a test and we'd use 1 if we wanted to actually have the computer make a move. Why would we do this? Because if the Function returns a value of 0 we know that there is no possible move and that the game is over. We'll look at that in a moment as it's the heart and soul of our game, but next let's look at how we'll accept input from our human player.

Not surprisingly we'll call a Function called humanplayer(), so let's see what that looks like.

```
Function humanplayer(turn)

        done=0
        //Find the square under the cursor
        findsquare()

        //Find out have many piece the player will win
        howmany=scansquare(squareacross,squaredown,turn)

        //It would be useful to let the player know if the
        //Move is valid, let's change the marker sprite colour to red
        //For in valid moves
        //We can tell this if the howmany search is 0
        If howmany=0
                //Let's set the marker sprite to red
                SetSpriteColor(marker,255,0,0,255)
                //If it's not 0, we'll set it back to it's normal colour
                Else
                SetSpriteColor(marker,255,255,255,255)
        EndIf

        If howmany<>0 and GetRawMouseLeftPressed()
                //Change the pieces
                changesquares(squareacross,squaredown,turn)
                //Place the players actual move
                board[squareacross,squaredown].side=turn
                //Flag as completed
                done=1
        EndIf

        //Retain from function, returning done at the same time so we know
        //If the move is completed
EndFunction done
```

First, we create a Local variable called *done*. This will return true from this Function if the player has made a move, i.e. actually chosen a square to go in. As you can see, we call quite a few other Functions here, starting with a simple one which finds the square the mouse pointer is above.

```
Function findsquare()

        //Grab the mouse position but offset it by the position of
        //The board on screen.
        tempx=GetPointerX()-156
        tempy=GetPointerY()-28

        //Our mouse position can never be less than 0,
        //But our board starts at position 128 + a border width and depth of 28 for our board
        //Which comes to a total of 156,
        //We want to assume that mouse position 0 is the edge of our board, so we use a
little maths.
        //All squares are 90 pixels square and the screen is 1024 pixels wide, -156 which
equals 896.
        //So divide our temporary position by 90
        //And ROUND the number to remove any decimals.
```

```
        //This will produce a board position of between 0-9
        //Of course, our board is numbered between 1-8
        //So we'll add 1 to our result.
        squareacross=Round(tempx/90)+1
        squaredown=Round(tempy/90)+1
        //We now ignore any result that isn't between 1-8
        If squareacross<1 or squareacross>8
                squareacross=0
        EndIf

        If squaredown<1 or squaredown>8
                squaredown=0
        EndIf

        //Now let's calculate the actual position on the screen based
        //On the squares found
        //But only if it's a valid number (I.e. Not 0)
        //If it's a valid position, we place the marker at that point.
        //If the marker is off the board we hide it.

        tempx=156+Round(tempx/90)*90
        tempy=28+Round(tempy/90)*90

        If squareacross<>0 and squaredown<>0
                SetSpriteVisible(marker,1)
                SetSpritePosition(marker,tempx,tempy)
                else
                SetSpriteVisible(marker,0)
        EndIf


EndFunction
```

There's nothing much new in the above snippet, so you will learn most from simply reading the comments throughout the Function. We do use two commands that you might not recognise and those are GetPointerX() and GetPointerY(), these simply return the position of the mouse pointer on screen.

Let's go back to our humanplayer() Function.

Next to be looked at is:

```
//Find out have many pieces the player will win
howmany=scansquare(squareacross,squaredown,turn)
```

So, let's look at this Function and it's quite a big one.

```
Function scansquare(across,down,whichside)

//We enter the Function with the following information.
//The square across we want to look at
//The square down we want to look at
//Which side is taking a turn

//Let's make sure that all our board spaces are unflagged
//We do this to clear any information retained from the last search
clearflagged()

//We set a local variable to indicate whether we're looking
//To capture Black (2) or White (3)
//We set it to Black at first
lookingfor=2
//But if it's Blacks turn
if whichside=2
//We set it to White.
lookingfor=3
endif


//If the search square is outside the board just exit as an invalid move
if across=0 or down=0
        //We return 0 here as no pieces can be captured
```

```
        ExitFunction 0
Endif


//Is the square empty?
//Only search if that's the case
if board[across,down].side=0

//Set a local variable to count the number of
//Pieces we might be able to capture
        found=0

        //Run through the search loop
        //We will be using our search array
        //Remember we search clockwise
        //Using 8 Directions.
        For look=1 to 8

                //Make sure we are using the selected square to
                //Start the search
                //We will also create 3 more Local variables
                //One to store the current search square
                //Across and Down
                //And a temporary variable tfound
                //Which will store the pieces captured in this search
                tacross=across
                tdown=down
                tfound=0

                //Start the search
                Do

                        //Adjust the search position
                        //So we're looking at the next available square
                        Inc tacross,search[look].across
                        Inc tdown,search[look].down


                //Exit the seach loop for this direction if the search has left the board
                //As we can't capture pieces if we are outside of the board
                If tacross<1 or tacross>8 or tdown<1 or tdown>8
                //Make 0 found as this is an invalid search
                        tfound=0
                        //Exit to the command following the Loop
                        Exit
                EndIf

                //Have we found a piece we can potentially capture?
                If board[tacross,tdown].side=lookingfor
                        //If so, increase the number found and flag the piece
                        //If so, we increase tfound by 1 indicating we've found another
                        //Piece to capture.
                        Inc tfound,1
                        //And we flag it as one that we might need to change.
                        board[tacross,tdown].flagged=1
                Endif


                //Has the seach run into an empty square
                //If it has this is another invalid search
                //As we haven't trapped a piece
                If board[tacross,tdown].side=0
                //Make 0 found as this is an invalid search
                        tfound=0
                        //Exit to the command following the Loop
                        Exit
                EndIf

                //If we find a piece that is the current
                //Players we can end our seach as valid

                If board[tacross,tdown].side=whichside
                        //Exit to the command following the Loop
                        //tfound will contain the value for this search
                        Exit
                Endif


                //Keep searching
                Loop
```

```
                //If none found ensure that all flagged peices are cleared for this search
                //But only in the last direction checked as the search
                //Found no valid pieces to capture.
                If tfound=0
                //Reset the search position to the square
                        tacross=across
                        tdown=down
                        //Start a new loop
                        Do

                        //adjust the search position
                         Inc tacross,search[look].across
                         Inc tdown,search[look].down
                        //Exit if it leaves the board
                         If tacross<1 or tacross>8 or tdown<1 or tdown>8
                                //Exit to the command following the Loop
                                Exit
                        EndIf

                        //Unflag this piece
                        board[tacross,tdown].flagged=0

                        //Exit if the square is empty or it contains one of the current
players
                        //Pieces as we know the search is over.

                        If board[tacross,tdown].side=0 or board[tacross,tdown].side=whichside
                                //Exit to the command following the Loop
                                Exit
                        EndIf


                        //Contine with the loop
                                Loop
                        EndIf

                        //Add the number of peices found in the search to the total.
                        //If this is 0, found won't be increased
                        inc found,tfound

                        //Finish searching in all 8 directions.
                Next look
        //End the search only if square empty If Loop.
        EndIf

        //Exit this Function, returning the found value,
        //Found is the maximum number of pieces that can be captured.
EndFunction found
```

Once again, there is only one new command included in this code, so you are best reading through the code, taking note of the comments. The new command is ExitFunction, works in a similar way to Exit, which we use to jump out of loop structures such as Do – Loop, While – EndWhile, etc. In this case, instead of jumping out of one of these it will completely exit the Function. In many ways this can be thought of as an early exit version of EndFunction, and is treated in the same way. If your Function returns a value, ExitFunction must also return a value.

So, we've scanned our square, now what? Well, as we've discussed our *scansquare* Function returns a value. 0 if the square can capture nothing or the number of tiles. This is where we change the colour of our marker, which you will remember is the colour of the human player if valid or dark red otherwise.

```
//It would be useful to let the player know if the
//Move is valid, let's change the marker sprite colour to red
//For in valid moves
//We can tell this if the howmany search is 0
        If howmany=0
                //Let's set the marker sprite to red
                SetSpriteColor(marker,255,0,0,255)
                //If it's not 0, we'll set it back to it's normal colour
                Else
                SetSpriteColor(marker,255,255,255,255)
```

```
        EndIf
```

So, we know if our move is valid, so now let's make the change.

```
If howmany<>0 and GetRawMouseLeftPressed()
        //Change the pieces
        changesquares(squareacross,squaredown,turn)
        //Place the players actual move
        board[squareacross,squaredown].side=turn
        //Flag as completed
        done=1
EndIf
```

Only one new command there, and that's `GetRawMouseLeftPressed()`, we're sure you won't be at all surprised when will tell you this will return True if the left mouse button has been pressed.

Now let's checkout *changesquares(turn)*

```
Function changesquares(whichside)

        //Scan the entire board
        for across=1 to 8
                for down=1 to 8
                        //If a square has been flagged for change
                        if board[across,down].flagged<>0
                                //Set the sprite the image for whichside
                                SetSpriteImage(board[across,down].sprite,whichside)
                                //Set the array to relect this.
                                board[across,down].side=whichside
                        endif
                next down
        next across

        //That's it.

EndFunction
```

Once again, there are no new commands and you should be able to understand what is going on here. If you don't, read back through the earlier sections and hopefully all will become clear.

So, we've made our move, all that remains is to jump out of `humanplayer()` returning a 1 if the move was valid, or a 0 if nothing has happened yet.

```
EndFunction done
```

Now, let's return to our main game loop. All that remains to do is to see if the human move was valid or if not, confirm if the human can move at all.

```
if hasmoved=1 or test=0
        setimages()
        counttiles()
        Sync()
        Exit
        Endif

        Sync()

 loop
```

So, *hasmoved* which was returned by the `Function humanplayer()` and test which was returned by the `Function computerplayer(?,?)` both have a possibility of ending this turn.

Note how we call `Sync()` a couple of times here, once before we exit the human player loop and

again if we haven't moved but can. We do this as the `Exit` will bypass the second `Sync()` if the player has finished or can't move, which will cause the screen to retain an image before the update. If this seems confusing, try removing the `Sync()` before the `Exit` command and try to work out what is happening.

If hasmoved<>0 then the human player has made a valid move.

If test=0, then it is impossible to capture anything, so the game is over.

In either case we update the images using `setimages()` which we've already discussed. We count the pieces for both players to see who is currently winning. To do this we call `counttiles()`

```
Function counttiles()

        player[1].score=0
        player[2].score=0

        for across=1 to 8
                for down=1 to 8
                        If board[across,down].side<>0
                                inc player[board[across,down].side-1].score,1
                        EndIf
                next down
        next across

        SetTextString(1,str(player[1].score))
        SetTextString(2,str(player[2].score))

EndFunction
```

Everything in this Function has been covered in this or past tutorials.

EXERCISE: *If you are in a group, discuss how and why this works. If you're learning this on your own, read through the above code and if you don't understand anything, read back through this and previous tutorials, and / or use the F1 help function until you can understand this code.*

Okay, we're nearly there.

Now let's take a look at the `computerplayer(?,?)`

In our main game loop, everything by the computer is handled here.

```
//If the player is the computer take it's turn
        if player[whoseturn-1].iscomputer=1
                //Flag the computers turn as 'real'
                //We will also return the pieces captured
                //In case the computer could not go.
                test=computerplayer(whoseturn,1)
                setimages()
                counttiles()
                Sync()
        EndIf
```

This little loop is very similar to the human player loop, but let's look at how the computer thinks when `computerplayer(whoseturn,1)` is called.

```
Function computerplayer(turn,real)

        //Reset some local variables
        //Best move will contain a value
```

```
        //Based on the number of pieces that can be captured
        //In addition, stratgic squares will be given more value
        //At higher levels of difficulty.
        bestmove=0
        //The square which provides the best move
        //Will be stored here.
        bestacross=0
        bestdown=0

        //So, we search our grid.
        for across=1 to 8
        for down=1 to 8
                //Is the square available for a possible move?
                //I.e. Is it empty.
                If board[across,down].side=0
                        //Scan the square
                        howmanyfound=scansquare(across,down,turn)
                        //Can anything be captured
                        If howmanyfound<>0

                        //The corners are more important as they are harder to capture
                        //So, add to howmany found to give them more value.
                        //But only if howdifficult=3 and this is a real move
                        If (across=1 and down=1) or (across=8 and down=1) or (across=1 and
                                down=8) or (across=8 and down=8) and real=1
                        //But only if we are on the highest level of difficulty
                        If howdifficult=3
                                //We increase howmanyfound by 4
                                //Giving the square more importance.
                                Inc howmanyfound,4
                        Endif
                        EndIf

                        //The edges are also important, but less so,
                        //But only if howdifficult is greater than 1
                        //And only if this is a real move.
                        If (across=1 or across=8 or down=1 or down=8) and real=1
                        //But only if not on the default, easy level
                                If howdifficult>1
                                //We increase howmanyfound by 2
                                //Giving the square more importance.
                                Inc howmanyfound,2
                                EndIf
                        EndIf

                        //Has the number of pieces that can be captured
                        //Plus the importance of any stategic square
                        //Exceeded the value of bestmove?
                        If howmanyfound>bestmove
                                //If so, make the new bestmove = howmanyfound
                                bestmove=howmanyfound
                                //Mark the squares where this move was found
                                bestacross=across
                                bestdown=down
                                EndIf
                        EndIf
                        EndIf
                        //Finish our search
                next down
        next across

        //If the search was real and not just a count
        //Make the move.
        If real=1
                If bestmove<>0
                        //reflag the peices for the computers chosen move
                        howmanyfound=scansquare(bestacross,bestdown,turn)
                        //change over all of the squares
                        changesquares(turn)
                        //place the players actual piece
                        board[bestacross,bestdown].side=turn
                EndIf
        EndIf

        //Return the value of bestmove so we know this is a good square.

EndFunction bestmove
```

Again, there is nothing new here, read through the comments to gain an understanding of how this

all works. You should have enough information from the previous lessons to interpret the code. Once again, if you have any issues, read back or use the F1 help in your IDE.

So, we know how to make a move, we've seen how the computer thinks, and we've learnt to scan arrays and how to use Functions for multiple things, simply through the use of flags, but we still don't know who has one.

Some of you might have noticed this command after both players have moved.

```
Sleep(1000)
```

This is a new one and freezes the application for 1,000 milliseconds or 1 second. Why? Well, simply put, the computer thinks too quickly, and we need a delay to notice it has moved.

Try commenting this out and playing a game, it can get a little confusing. You might also want to try and make the sleep time longer, say Sleep(3000) – this can make the game annoying because you make the player wait longer.

We're nearly there, all we now need is a winner. There is actually only one way of knowing when the game is over, and that's when neither player can move. We now this from when we called `computerplayer(whoseturn,1)` or `computerplayer(whoseturn,0)` and assigned the result to test. If we return to our main game loop, let's check.

```
//If test=0 (returned from our game loops)
//No one can go, and the game is over.
If test=0

        If player[1].score>player[2].score
        Print("Black has won!")
        Endif

        If player[1].score<player[2].score
        Print("White has won!")
        Endif

        If player[1].score=player[2].score
        Print("The game is a draw!")
        Endif

Endif
```

Once more, we haven't commented this final code as we're leaving it to you to try to work out what is happening, but if you've been following these tutorials from the beginning we're confident you will have everything you need to understand it all.

EXERCISE: *Find a way of improving the basic difficulty of the game. We've already provided this for you, but you'll need to look for the variable you need to change to increase the challange.*

EXERCISE: *Read through the code and change the game in the following way: Make the computer play itself. Change the human player to White.*